

**ANTHONY BOILY**

Projet professionnel 2

420-VC2-LP

## **Rapport écrit ESP**

**Travail présenté à :**

**Michel Forget et Nicolas Demers**

Techniques de l'informatique

Cégep de La Pocatière

03 juin 2026

# Contents

|           |                                                                                                 |           |
|-----------|-------------------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Identification du projet</b>                                                                 | <b>3</b>  |
| <b>2</b>  | <b>Description sommaire du projet</b>                                                           | <b>3</b>  |
| <b>3</b>  | <b>Identification des domaines de l’informatique concernés</b>                                  | <b>4</b>  |
| <b>4</b>  | <b>Réalisation du projet</b>                                                                    | <b>4</b>  |
| 4.1       | Application de bureau téléchargeable . . . . .                                                  | 4         |
| 4.2       | Widget Spotify intégré . . . . .                                                                | 4         |
| 4.3       | Synchronisation Steam et données Deadlock . . . . .                                             | 5         |
| 4.4       | Système d’Awards . . . . .                                                                      | 5         |
| 4.5       | Game Overlay transparent . . . . .                                                              | 6         |
| 4.6       | Live Dashboard . . . . .                                                                        | 6         |
| <b>5</b>  | <b>Comment le projet a été réalisé</b>                                                          | <b>6</b>  |
| 5.1       | Méthodologie et outils de planification . . . . .                                               | 6         |
| 5.2       | Étapes de réalisation . . . . .                                                                 | 7         |
| 5.3       | Environnement de développement . . . . .                                                        | 7         |
| <b>6</b>  | <b>Technologies choisies</b>                                                                    | <b>8</b>  |
| 6.1       | Architecture Electron et communication IPC . . . . .                                            | 8         |
| 6.2       | Intégration OAuth — Spotify . . . . .                                                           | 9         |
| 6.3       | Python comme moteur de données et OCR . . . . .                                                 | 10        |
| 6.4       | Récapitulatif des technologies . . . . .                                                        | 11        |
| <b>7</b>  | <b>Nombre d’heures consacrées</b>                                                               | <b>11</b> |
| <b>8</b>  | <b>Ressources utilisées</b>                                                                     | <b>11</b> |
| <b>9</b>  | <b>Sources d’information</b>                                                                    | <b>12</b> |
| 9.1       | Fondations et environnement de développement . . . . .                                          | 12        |
| 9.2       | APIs et données de jeu . . . . .                                                                | 12        |
| 9.3       | Design, UX et inspiration . . . . .                                                             | 12        |
| 9.4       | Linux et overlay in-game . . . . .                                                              | 12        |
| <b>10</b> | <b>Difficultés rencontrées et solutions</b>                                                     | <b>13</b> |
| 10.1      | API communautaire instable et données dépréciées . . . . .                                      | 13        |
| 10.2      | Build CI/CD : CSP et variables d’environnement . . . . .                                        | 13        |
| 10.3      | Live Dashboard : limites du endpoint de parties actives . . . . .                               | 13        |
| 10.4      | OCR worker : le défi du roster live . . . . .                                                   | 13        |
| 10.5      | Awards non calculables : données absentes de l’API . . . . .                                    | 14        |
| <b>11</b> | <b>Limites de la réalisation, et pourquoi</b>                                                   | <b>14</b> |
| <b>12</b> | <b>Compétences du DEC réinvesties</b>                                                           | <b>15</b> |
| <b>13</b> | <b>Analyse de conception : Cycle de vie et cheminement de pensée de l’onglet Live Dashboard</b> | <b>16</b> |
| 13.1      | Phase 1 : Identification des contraintes d’API et architecture de contournement                 | 16        |

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| 13.2 Phase 2 : Design asynchrone et gestion d'état de l'interface . . . . . | 16        |
| 13.3 Phase 3 : Gestion défensive des cas d'erreur (Fallbacks) . . . . .     | 17        |
| <b>14 Bilan des apprentissages réalisés</b>                                 | <b>17</b> |
| 14.1 Apprentissages techniques . . . . .                                    | 17        |
| 14.2 Apprentissages en gestion de projet et outils . . . . .                | 22        |

## List of Figures

|    |                                                                                                                                                    |    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Bibliothèque de héros — vue principale de l'application . . . . .                                                                                  | 3  |
| 2  | Widget Spotify — contrôles de lecture et pochette d'album . . . . .                                                                                | 5  |
| 3  | Page Profil — système d'Awards avec raretés et statistiques . . . . .                                                                              | 5  |
| 4  | Game Overlay — superposition transparente par-dessus le jeu . . . . .                                                                              | 6  |
| 5  | Live Dashboard — grille des 12 joueurs avec statistiques . . . . .                                                                                 | 7  |
| 6  | Architecture multi-processus Electron — flux IPC et Context Bridge . . . .                                                                         | 8  |
| 7  | Diagramme de classes simplifié pour l'encapsulation et le traitement des entités du jeu. . . . .                                                   | 9  |
| 8  | Arbre de décision des awards basés sur les âmes par minute . . . . .                                                                               | 19 |
| 9  | Arbre de décision des awards Kill Participation — porte d'entrée absolue (kpActions) et seuils de pourcentage hiérarchisés. . . . .                | 19 |
| 10 | Diagramme de flux de l'approximation du killstreak — intervalles inter-morts et snapshots cumulatifs <code>stats[]</code> , reçus via IPC. . . . . | 20 |

# 1 Identification du projet

Mon projet consiste à développer une application compagnon et d'assistance en temps réel, pour optimiser l'expérience des joueurs et analyser leurs statistiques sur le jeu vidéo *Deadlock*, un jeu de tir tactique multijoueur en ligne développé par Valve.

## 2 Description sommaire du projet

*DeadlockHelper* est une application de bureau multiplateforme (Windows et Linux) conçue pour fonctionner en arrière-plan pendant que l'utilisateur joue à *Deadlock*. Elle est distribuée gratuitement à l'adresse <https://diredoch.github.io/DeadlockHelper/> sous forme d'un fichier `.AppImage` pour Linux et d'un installateur `.exe` pour Windows.

L'application est structurée autour de plusieurs onglets thématiques adaptés au cycle de vie du joueur. **Hors-partie**, elle sert de centre de connaissances : bibliothèque de héros, explorateur de *builds* d'objets recommandés par la communauté, et onglet *Profil* affichant l'historique de matchs assorti d'un système d'Awards calculés sur les performances réelles de l'utilisateur. **En partie**, un overlay transparent se superpose au jeu pour afficher des timers (Mid Boss, Urne) et des suggestions d'objets contre la composition adverse. **Post-partie**, le *Live Dashboard* regroupe les tuiles statistiques de tous les joueurs d'un match récent sur une grille de  $6 \times 2$ .

Un widget Spotify intégré permet de contrôler la musique (lecture, pause, piste suivante, rewind) sans quitter l'application. La synchronisation avec le compte Steam de l'utilisateur est requise pour récupérer son profil et ses données de matchs via l'API communautaire *Deadlock-API.com*.

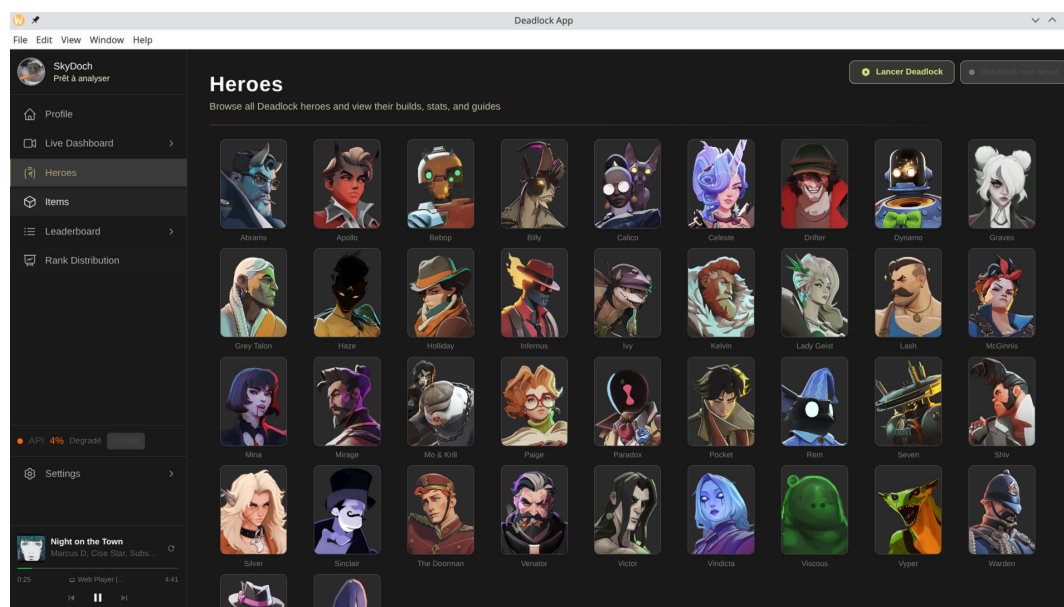


Figure 1: Bibliothèque de héros — vue principale de l'application

### 3 Identification des domaines de l'informatique concernés

Ce projet mobilise la majorité des domaines enseignés dans le cadre du DEC en Techniques de l'informatique. Le tableau suivant présente chaque domaine avec le cours associé et son application concrète dans le projet :

| Domaine                    | Cours      | Application dans le projet                  |
|----------------------------|------------|---------------------------------------------|
| Développement web          | 420-VBA-LP | Interface Electron, TypeScript, HTML/CSS    |
| Prog. web dynamique        | 420-V88-LP | Vite, routing, composants UI asynchrones    |
| Prog. orientée objet       | 420-V83-LP | Interfaces TypeScript, entités Player/Match |
| Systèmes d'exploitation    | 420-V89-LP | Processus Linux/Windows, fichiers config    |
| Sécurité et automatisation | 420-VB6-LP | Scripts CI/CD, packaging .AppImage/.exe     |
| Algorithmique              | 420-V80-LP | Calculs métriques, filtrage JSON volumineux |
| Programmation avancée      | 420-VA9-LP | Python 3.12, EasyOCR, traitement de données |
| Base de données            | 420-V91-LP | electron-store, cache local, TTL 7 jours    |
| Développement de jeux      | 420-VC3-LP | Contexte Deadlock, overlay in-game          |
| Gestion de projet          | 420-VB9-LP | Kanban, GitHub, planification Agile         |
| Mathématiques appliquées   | 201-V03-LP | Pondération statistique des Awards          |
| Algèbre et statistiques    | 201-V04-LP | Seuils de performance, percentiles          |

## 4 Réalisation du projet

### 4.1 Application de bureau téléchargeable

L'application est distribuée via GitHub Pages. La compilation est automatisée par GitHub Actions : à chaque version taguée, le pipeline produit un `.AppImage` autosuffisant pour Linux et un installateur `.exe` pour Windows. L'utilisateur n'a aucun prérequis à installer manuellement.

### 4.2 Widget Spotify intégré

Le widget Spotify permet de contrôler la musique directement depuis l'application : lecture, pause, piste suivante et piste précédente. L'authentification repose sur le flux *Authorization Code with PKCE* de l'API Spotify Web : aucun secret client n'est stocké dans le code compilé. Les jetons d'accès (valides 1 heure) et de rafraîchissement sont

persistés localement.

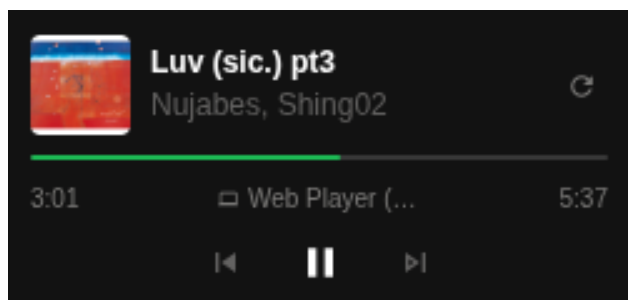


Figure 2: Widget Spotify — contrôles de lecture et pochette d’album

### 4.3 Synchronisation Steam et données Deadlock

L’authentification Steam utilise le protocole OpenID, qui redirige l’utilisateur vers la page officielle de Valve sans jamais exposer son mot de passe à l’application. Une fois connecté, l’application récupère le **SteamID64** et interroge *Deadlock-API.com* pour l’historique de matchs, les statistiques par héros et le rang.

### 4.4 Système d’Awards

Le système d’Awards analyse l’historique de matchs et attribue des médailles de performance. Il regroupe 48 awards en 5 raretés : *Common*, *Uncommon*, *Rare*, *Epic* et *Infamous*. Chaque award est calculé à partir du champ `stats[]` de l’API (données de fin de match) : dégâts par minute, âmes par minute, taux de participation aux éliminations, précision de tir, etc.

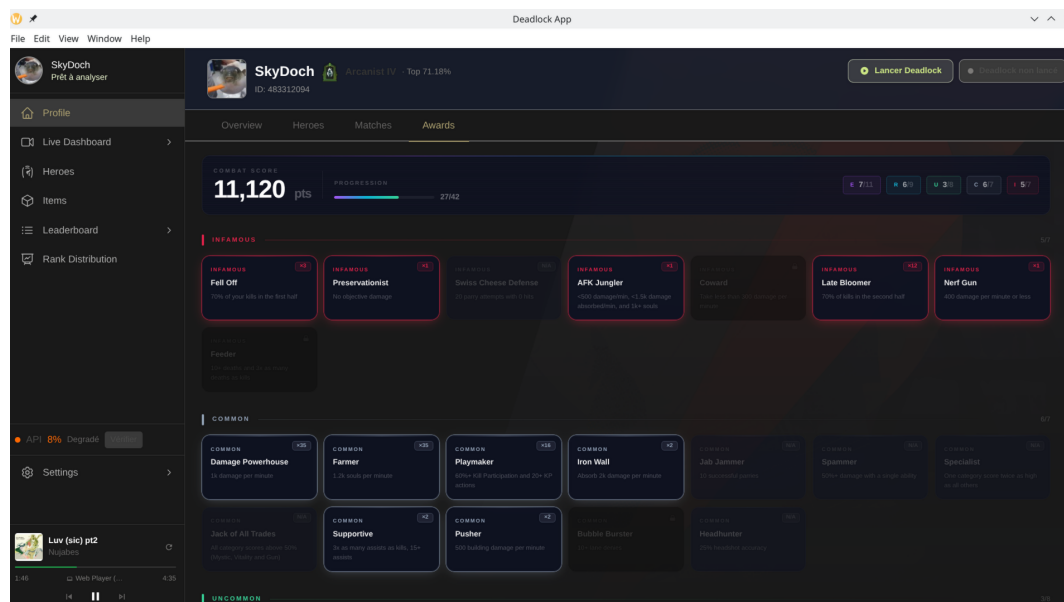


Figure 3: Page Profil — système d’Awards avec raretés et statistiques

## 4.5 Game Overlay transparent

L'overlay in-game est une seconde fenêtre Electron (`BrowserWindow`) configurée avec `frame: false`, `transparent: true` et `alwaysOnTop: true`. Elle se superpose au jeu sans perturber son déroulement, avec quatre composants : un timer Mid Boss (déclenché manuellement), un timer d'Urne (déterministe, calculé depuis l'horloge de partie), des suggestions d'objets contre la composition ennemie, et un indicateur Souls/min (placeholder).



Figure 4: Game Overlay — superposition transparente par-dessus le jeu

## 4.6 Live Dashboard

Le Live Dashboard affiche les 12 joueurs d'un match sous forme de grille. Il récupère le roster via le *bulk metadata endpoint* de l'API (`GET /v1/matches/metadata?match_ids={id}`), qui tolère 10 requêtes par minute sans dépendance Steam, contrairement au endpoint unitaire limité à 3 requêtes par heure.

# 5 Comment le projet a été réalisé

## 5.1 Méthodologie et outils de planification

Le projet a été planifié et suivi avec **Microsoft Planner** selon un tableau Kanban. Chaque fonctionnalité a été découpée en tâches étiquetées selon leur priorité : *MVP* (ce qui doit absolument figurer dans la démonstration finale) ou *Backlog* (améliorations à venir). Cette distinction a permis de livrer un produit fonctionnel dans les délais tout en gardant une vision claire des fonctionnalités futures.

En parallèle, un vault **Obsidian** a servi de base de connaissances personnelle, avec notamment un diagramme *Excalidraw* dessiné sur 1 à 2 semaines avant le début du code pour visualiser l'architecture globale, la navigation entre onglets et les flux de données entre les API. Cette étape s'est révélée cruciale pour anticiper les dépendances et les risques techniques.



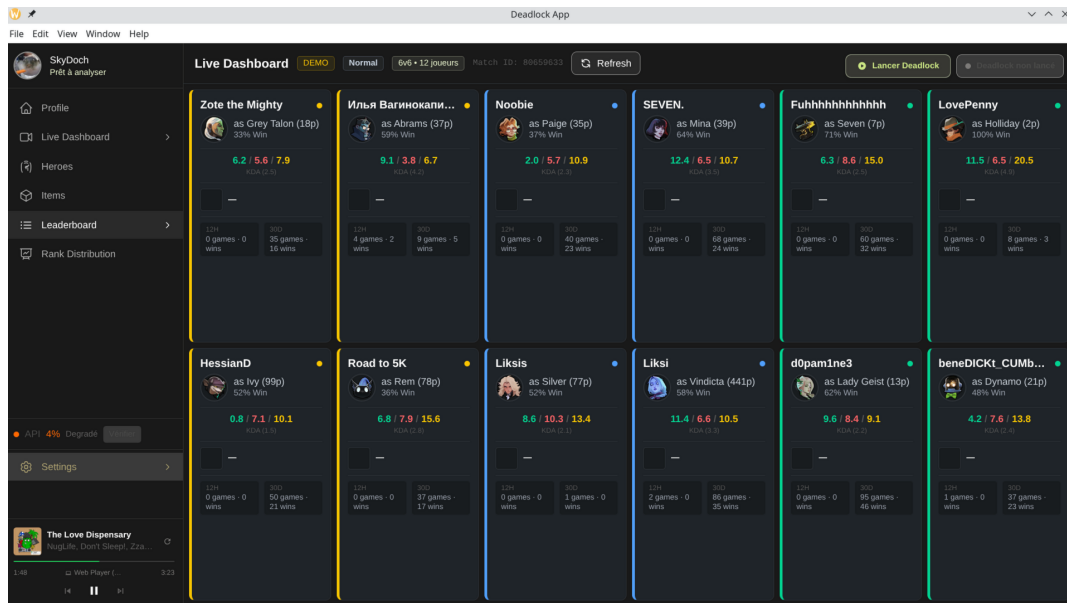


Figure 5: Live Dashboard — grille des 12 joueurs avec statistiques

Un **PRD** (*Product Requirement Document*) a été rédigé pour recenser toutes les sources externes nécessaires : clés API à obtenir, portails développeur à consulter, contraintes d'authentification à respecter.

## 5.2 Étapes de réalisation

1. **Planification Kanban** : découpage des fonctionnalités en tâches individuelles, étiquetage MVP vs. Backlog dans Microsoft Planner.
2. **Prototypage visuel** : modélisation de l'architecture et des écrans dans Excalidraw (vault Obsidian) pour valider la vision globale avant de coder.
3. **PRD** : recensement exhaustif des API, portails développeur, clés et contraintes d'accès.
4. **Validation des technologies risquées en premier** : authentification Steam (OpenID) et Spotify (OAuth PKCE), heartbeat de disponibilité de l'API Deadlock — prouver que le *stack* fonctionne avant d'investir dans l'interface.
5. **Construction de l'interface** : charte de couleurs définie en CSS, arborescence de fichiers structurée, navigation fonctionnelle, pages marquées « En cours de développement » comme placeholders.
6. **Développement itératif** : implémentation des fonctionnalités par ordre décroissant de risque et de complexité, avec commits réguliers et documentation des décisions architecturales (ADR).

## 5.3 Environnement de développement

Le développement s'est déroulé sur deux systèmes : Windows avec l'IDE *Cursor* (version de VS Code enrichie de fonctionnalités d'IA générative) et Linux CachyOS avec *Code OSS* (version open-source de VS Code). *NeoVIM* a été expérimenté comme éditeur secondaire. Git et GitHub ont servi pour la gestion de version et le CI/CD.

## 6 Technologies choisies

### 6.1 Architecture Electron et communication IPC

Le cœur de l'application repose sur **Electron 40.0.0** couplé à **Vite** et **TypeScript**. Electron permet de créer des applications de bureau avec les technologies web tout en donnant accès aux API système (fichiers, processus, réseau). Son architecture est **multi-processus** : un processus principal (`main.ts`) gère les fenêtres et les appels système, un ou plusieurs processus de rendu constituent l'interface graphique. La communication entre les deux passe par un canal sécurisé appelé **IPC** (*Inter-Process Communication*).

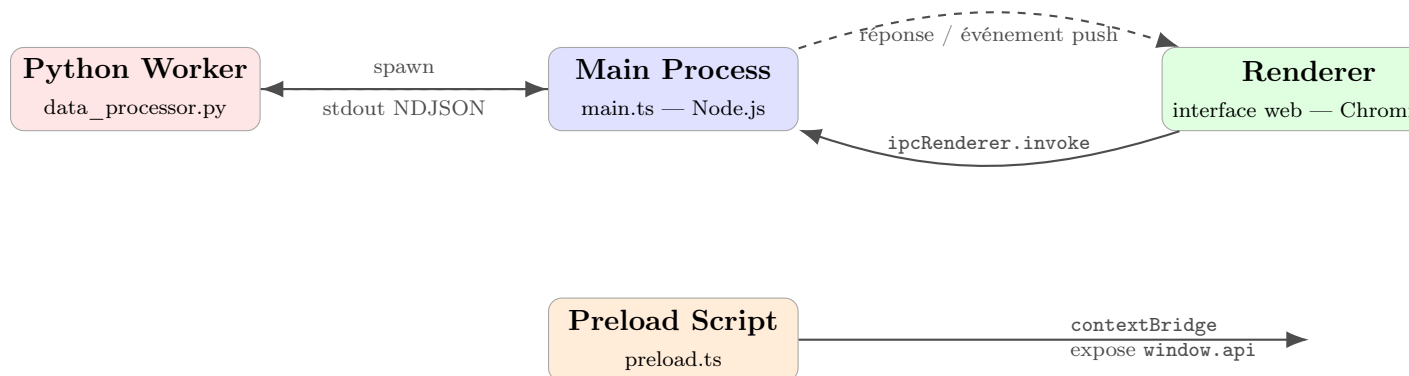


Figure 6: Architecture multi-processus Electron — flux IPC et Context Bridge

Le **Context Bridge** est le pont de sécurité entre le preload et le renderer. Il expose uniquement les fonctions autorisées via `window.api`, sans jamais donner au renderer un accès direct à `Node.js`.

```

1 contextBridge.exposeInMainWorld('api', {
2   // Requete generique vers le processus principal
3   request: (endpoint: string, options?: { method?: string; body?: any
4     }) =>
5     ipcRenderer.invoke('api:request', { endpoint, ...options }),
6
7   // Authentification Steam (OpenID)
8   steamStartAuth: () =>
9     ipcRenderer.invoke('steam:startAuth'),
10
11  // Disponibilite de l'API Deadlock (heartbeat)
12  getApiAvailability: () =>
13    ipcRenderer.invoke('api:get-availability'),
14
15  // Push du main vers renderer : alertes de sante de l'API
16  onHealthStatusChange: (callback: (availability: number) => void) =>
17    {
18      ipcRenderer.on('api:health-alert', (_event, availability: number)
19        => {
20        callback(availability);
21      });
22    },
23  },
24  });
  
```

Listing 1: Extrait de `preload.ts` — exposition sécurisée via `contextBridge`

La figure 7 représente les principales entités manipulées par l’application sous forme de diagramme de classes UML simplifié. Cette modélisation orientée objet reflète directement les interfaces TypeScript strictement typées qui structurent la communication inter-processus : chaque entité (`MatchData`, `PlayerMeta`, `DynamicTagManager`, `AwardEvaluator`) est encapsulée dans une interface immuable, et seuls les champs explicitement déclarés transitent par les canaux IPC. Cette conception garantit qu’aucune donnée brute ou objet complexe non sérialisable ne peut corrompre la communication entre le processus principal et le renderer Chromium, tout en offrant une auto-documentation naturelle du contrat de données à chaque frontière architecturale.

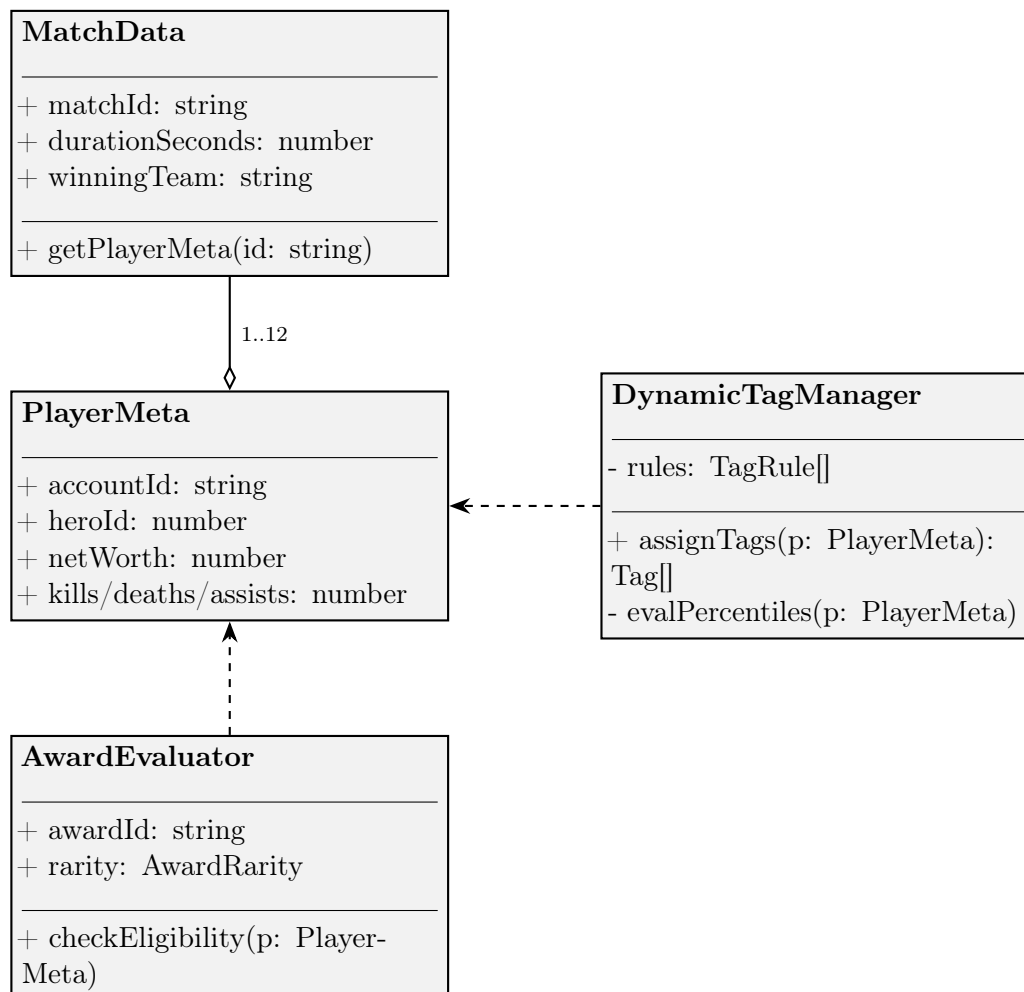


Figure 7: Diagramme de classes simplifié pour l’encapsulation et le traitement des entités du jeu.

## 6.2 Intégration OAuth — Spotify

L’authentification Spotify utilise le flux *Authorization Code with PKCE*. Un serveur HTTP local temporaire sur le port 30765 réceptionne le callback OAuth. Pour résoudre l’injection des clés API dans le build compilé, un système de credentials en fichier JSON (placé dans le dossier `userData` d’Electron) a été mis en place :

```

1 function loadCredentials() {
2   // Chemin : ~/.config/DeadlockHelper/spotify-credentials.json (
   Linux)

```

```

3 // ou %APPDATA%\DeadlockHelper\spotify-credentials.json (
4 // Windows)
5 const configPath = path.join(app.getPath('userData'),
6                               'spotify-credentials.json');
7 if (fs.existsSync(configPath)) {
8   const raw = JSON.parse(fs.readFileSync(configPath, 'utf8'));
9   return {
10     clientId:    raw['clientId'],
11     clientSecret: raw['clientSecret'],
12     redirectUri:  raw['redirectUri'],
13   };
14 }
15 // Repli sur les valeurs injectees au build (environnement de dev)
16 return {
17   clientId:    process.env.SPOTIFY_CLIENT_ID    ?? '',
18   clientSecret: process.env.SPOTIFY_CLIENT_SECRET ?? '',
19   redirectUri: process.env.SPOTIFY_REDIRECT_URI
20               ?? 'http://127.0.0.1:30765/spotify/callback',
21 };
22 }

```

Listing 2: Extrait de spotify-logic.ts — chargement dynamique des identifiants

### 6.3 Python comme moteur de données et OCR

Un processus Python 3.12 tourne en parallèle d'Electron. Il gère le traitement des réponses JSON volumineuses de l'API et le worker OCR (EasyOCR + thefuzz). Il communique avec Electron exclusivement via `stdout` (JSON structuré) et `stderr` (diagnostics), sans port réseau.

```

1 # Vocabulaire ferme des heros (hero_name -> hero_id officiel)
2 # Source : GET https://api.deadlock-api.com/v1/assets/heroes
3 HERO_MAP = {
4   "Abrams": 6,  "Bebop": 15, "Billy": 63,  "Calico": 52,
5   "Dynamo": 11, "Haze": 17,  "Infernus": 1, "Ivy": 20,
6   "Kelvin": 12, "Lash": 16,  "McGinnis": 18,
7   "Mo and Krill": 10, "Paradox": 22, "Pocket": 50,
8   "Seven": 4,   "Shiv": 51,  "Warden": 27, "Wraith": 8,
9   # ... 28 heros au total
10 }
11
12 # Sortie NDJSON sur stdout (une ligne JSON par scan)
13 # Diagnostics sur stderr pour ne pas polluer le flux JSON
14 result = {"myTeam": my_team, "enemyTeam": enemy_team}
15 print(json.dumps(result), flush=True) # lu par ipcMain d'Electron

```

Listing 3: Extrait de ocr-worker/main.py — table des héros et sortie NDJSON

## 6.4 Récapitulatif des technologies

| Technologie              | Rôle dans le projet                               |
|--------------------------|---------------------------------------------------|
| Electron 40.0.0 + Vite   | Framework de bureau, bundler                      |
| TypeScript               | Langage principal (main, renderer, preload)       |
| Python 3.12              | Traitement de données, OCR                        |
| Tailwind CSS + ShadCN UI | Design system, composants UI                      |
| electron-store           | Persistance locale (tokens, cache matchs, awards) |
| GitHub Actions           | CI/CD — compilation .AppImage et .exe             |
| Deadlock-API.com         | Données de matchs, héros, rang                    |
| Steam Web API (OpenID)   | Authentification utilisateur                      |
| Spotify Web API (PKCE)   | Contrôle de la lecture musicale                   |
| UptimeRobot              | Monitoring heartbeat de l'API                     |
| EasyOCR + thefuzz        | Reconnaissance de texte à l'écran (OCR)           |

## 7 Nombre d'heures consacrées

Au départ du projet, une charge de travail de **150 heures** avait été estimée pour l'ensemble du développement, de la planification à la démonstration finale. En réalité, le projet a nécessité environ **175 heures**, soit un dépassement de 25 heures. Ce surplus est principalement attribuable à trois facteurs imprévus : l'instabilité de l'API communautaire (endpoints qui changent, données dépréciées à filtrer), les problèmes de build CI/CD (CSP, injection des variables d'environnement), et le temps investi dans l'exploration de solutions alternatives pour le Live Dashboard et le worker OCR.

## 8 Ressources utilisées

### Claude Code (Anthropic)

Abonnement à 30\$/mois, utilisé durant le dernier mois de développement intensif pour l'assistance au code, la génération de documentation et la rédaction du présent rapport. Non prévu dans le budget initial ; dépense personnelle pour l'éducation.

### Spotify Premium

Abonnement à 12\$/mois pendant 5 mois (60\$ au total). Requis pour tester l'intégration du widget : les fonctionnalités de contrôle de l'API Spotify nécessitent un compte Premium actif. Cette ressource était incluse dans le budget initial estimé.

### Microsoft Planner

Outil de gestion de projet (Kanban) inclus dans la suite Microsoft 365 du Cégep. Aucun coût supplémentaire.

### GitHub Actions

CI/CD pour la compilation et la distribution automatisées. Gratuit dans le cadre du plan public GitHub.

**Matériel** Ordinateur personnel (dual-boot Windows 11 / CachyOS Linux) et un second écran pour tester l'application en conditions réelles de jeu (overlay in-game).

## 9 Sources d'information

### 9.1 Fondations et environnement de développement

- **Electron.js** (<https://www.electronjs.org/>) — Framework de référence pour créer des applications de bureau avec des technologies web. Point d'entrée principal de l'architecture.
- **Electron 40 — Timeline** (<https://www.electronjs.org/fr/docs/latest/tutorial/electron-timelines>) — Consulté pour le choix de la version stable et la politique de support à long terme.
- **Python 3.12** (<https://docs.python.org/3.12/>) — Référence officielle pour le moteur de données et le worker OCR.
- **Next.js** (<https://nextjs.org/docs>) — Consulté pour les meilleures pratiques de routage et de performance, appliquées au renderer.
- **PEP 257** (<https://peps.python.org/pep-0257/>) — Convention de documentation des docstrings Python, appliquée à `main.py` et `data_processor.py`.

### 9.2 APIs et données de jeu

- **Deadlock API** (<https://assets.deadlock-api.com/scalar> et <https://deadlock-api.com/>) — Source principale de données : historique des matchs, statistiques des héros, rang, scouting de joueurs.
- **UptimeRobot** (<https://dashboard.uptimerobot.com/monitors/802150090>) — Monitoring de la disponibilité de l'API. Utilisé pour le heartbeat intégré à l'application (vérification toutes les 5 minutes).
- **Steam Web API** (<https://steamcommunity.com/dev>) — Authentification OpenID et vérification de l'installation locale du jeu (`libraryfolders.vdf`).
- **Spotify Web API** (<https://developer.spotify.com/documentation/web-api>) — Guide technique complet pour l'intégration du lecteur multimédia et le flux OAuth PKCE.

### 9.3 Design, UX et inspiration

- **Tailwind CSS** (<https://tailwindcss.com/>) et **ShadCN UI** (<https://ui.shadcn.com/>) — Design system utilisé pour construire une interface moderne, performante et cohérente.
- **Porofessor.gg** (<https://porofessor.gg/fr/>) et **U.gg** (<https://u.gg/>) — Sites de référence du genre *companion app* sur League of Legends, utilisés comme modèles pour l'organisation visuelle des statistiques et la création de tags clairs.

### 9.4 Linux et overlay in-game

- **gtk-layer-shell** (<https://github.com/wmww/gtk-layer-shell>) — Documentation sur les layers Wayland pour les fenêtres overlay.

- **MangoHud** (<https://github.com/flightlessmango/MangoHud>) — Référence open-source d'overlay pour jeux Linux, consulté pour les bonnes pratiques de composition de fenêtres.

## 10 Difficultés rencontrées et solutions

### 10.1 API communautaire instable et données dépréciées

*Deadlock-API.com* est une API communautaire non officielle, encore en développement actif. Les noms d'endpoints changent, des héros retirés du jeu restent dans les données, et certaines images pointent vers des URLs invalides. La solution a été de maintenir une logique de filtrage côté application : ignorer les héros dont le champ `disabled` est `true`, filtrer les images qui retournent une erreur 404, et mettre en cache les données valides pour limiter les appels inutiles.

### 10.2 Build CI/CD : CSP et variables d'environnement

Deux problèmes majeurs ont bloqué le pipeline GitHub Actions lors du passage en production :

1. **Content Security Policy (CSP)** : en mode production, Electron applique une CSP stricte qui bloquait l'affichage des images hébergées sur des domaines externes (pochettes Spotify sur `i.scdn.co`, avatars Steam, icônes de héros). La solution a été d'implémenter une double couche CSP : une déclarée dans le processus principal via `session.defaultSession.webRequest.onHeadersReceived`, et une dans les entêtes HTML, en autorisant explicitement chaque domaine externe utilisé.
2. **Variables d'environnement non injectées en production** : les clés API définies dans `.env` ne suivent pas le build compilé. La solution a été de créer un fichier JSON de credentials placé dans le dossier `userData` d'Electron, accessible à l'exécution sans être embarqué dans le binaire distribué.

### 10.3 Live Dashboard : limites du endpoint de parties actives

Le premier plan pour le *Live Dashboard* reposait sur l'endpoint `/matches/active`. En pratique, il est limité aux 200 parties les plus regardées (parties de haut rang ou en tournoi) : les parties normales n'y apparaissent jamais. De plus, le endpoint unitaire `/v1/matches/{id}/metadata` est limité à 3 requêtes par heure et dépend de Steam (retourne des erreurs 503 intermittentes).

La solution retenue a été de basculer vers le *bulk metadata endpoint* (`GET /v1/matches/metadata?match_` qui offre un quota de 10 requêtes par minute sans dépendance Steam. Un normaliseur a été développé pour convertir son schéma vers le format attendu par l'interface (les champs `team`, `game_mode` et `winning_team` diffèrent entre les deux endpoints).

### 10.4 OCR worker : le défi du roster live

Récupérer les noms et héros de chaque joueur *pendant* une partie s'est avéré être le défi technique le plus complexe. Trois approches ont été tentées en séquence :

1. **Fichier console.log (-condebug) :** le log de Deadlock ne contient que les informations du compte connecté et l'identifiant de la partie. Les noms et héros des autres joueurs n'y apparaissent pas.
2. **Screenshot + OCR :** un worker Python (EasyOCR + thefuzz) lit le panneau « ESC > PLAYERS » de Deadlock. **Résultat : 12/12 héros identifiés (100 %)** et 10/12 pseudos Steam. Limite fatale : un pseudo Steam affiché en jeu n'est pas résolvable en `account_id` (aucun endpoint Steam ou Deadlock ne supporte cette recherche), rendant les statistiques par joueur impossibles.
3. **Attente post-partie :** solution finale adoptée. Le Live Dashboard attend que la partie soit ingérée par l'API (quelques minutes après la fin), puis affiche les données complètes. Un mode MOCK dans les Paramètres permet de simuler un match en cours avec des données historiques.

## 10.5 Awards non calculables : données absentes de l'API

Sur les 48 awards définis, 21 ne peuvent pas être calculés : les données nécessaires sont absentes du payload `CMsgMatchMetaDataContents`, vérifié sur 153 clés distinctes d'une vraie réponse API. Les champs manquants sont : taux de tirs à la tête, compteurs de parries, dégâts d'antiheal, dégâts par capacité individuelle et scores de catégories. Ces 21 awards affichent « Données indisponibles » dans l'interface, avec une explication explicite pour l'utilisateur.

# 11 Limites de la réalisation, et pourquoi

## Roster en temps réel (Live Dashboard)

L'API Deadlock n'expose les données d'une partie qu'après sa conclusion (ingestion post-partie). Il est donc impossible d'afficher les statistiques des joueurs *pendant* une partie active. Un mode MOCK compense partiellement en permettant de simuler un match en cours avec des données historiques.

## Overlay en mode fullscreen exclusif

La propriété `alwaysOnTop` d'Electron ne passe pas au-dessus d'un jeu en mode fullscreen exclusif sous Linux/Wayland. L'overlay n'est visible que si le jeu tourne en mode *Borderless Windowed*. Ce prérequis est documenté dans les Paramètres de l'application et vérifié au démarrage.

## Composant Souls/min en overlay

La quantité d'âmes ramassées par le joueur n'est disponible dans aucune source accessible en temps réel : ni le `console.log`, ni l'API active match, ni un GSI officiel de Valve. Le composant affiche « - SPM » comme placeholder honnête.

## OCR worker non intégré à Electron

Le module Python EasyOCR fonctionne correctement en standalone (validé à 12/12 héros), mais son intégration dans Electron (spawn/kill du processus, lecture du flux NDJSON, toggle dans les Paramètres) n'a pas été finalisée. Même intégré, la limite de résolution des pseudos Steam en `account_id` réduirait l'utilité du module aux seules données de héros.



## 21/48 awards indisponibles

L'API ne fournit pas les données pour 21 des 48 awards. Cette limite est indépendante de l'application et dépend d'une éventuelle mise à jour de l'API communautaire ou d'une API officielle Valve.

## 12 Compétences du DEC réinvesties

Ce projet constitue l'aboutissement de ma formation et mobilise l'ensemble des compétences acquises tout au long du DEC en Techniques de l'informatique. Il ne s'agit pas uniquement d'écrire du code, mais d'orchestrer plusieurs technologies pour répondre à un besoin utilisateur concret, du même type que ceux rencontrés en milieu professionnel.

**Développement logiciel et web (420-VBA-LP, 420-V88-LP) :** l'interface construite avec Electron, Vite et TypeScript est une application directe des cours de développement web : gestion des événements asynchrones, manipulation du DOM, routage entre pages, gestion des états. L'utilisation de Tailwind CSS et ShadCN UI prolonge les bonnes pratiques de mise en page apprises en cours.

**Programmation orientée objet (420-V83-LP) :** la structure modulaire du code repose sur des interfaces TypeScript strictement typées (`AwardEntry`, `RichMatchMeta`, `GameState`, `OcrRoster`) qui encapsulent les données de chaque entité. Les principes de séparation des responsabilités et d'encapsulation sont appliqués systématiquement dans chaque module.

**Algorithmique et programmation avancée (420-V80-LP, 420-VA9-LP) :** le moteur Python traite des payloads JSON volumineux, calcule des métriques par minute, et implémente une approximation des séquences de kills basée sur l'analyse des intervalles temporels entre les morts.

**Systèmes d'exploitation (420-V89-LP) :** l'adaptation cross-platform (Windows ↔ Linux/Wayland) a exigé de maîtriser la gestion des processus, les conventions de chemins, la détection du répertoire Steam via `libraryfolders.vdf`, et les contraintes de composition de fenêtres sous Wayland.

**Sécurité et automatisation (420-VB6-LP) :** le pipeline CI/CD GitHub Actions automatise la compilation et la distribution. La gestion sécurisée des clés API (séparation build-time vs. runtime) et l'implémentation de la CSP relèvent directement de ce domaine.

**Base de données (420-V91-LP) :** `electron-store` joue le rôle de base de données locale : stockage des tokens, cache des métadonnées de matchs, persistance des awards et TTL de 7 jours sur les noms de joueurs.

**Mathématiques et statistiques (201-V03-LP, 201-V04-LP) :** les 27 awards calculables reposent sur des métriques quantitatives (ratios, seuils, percentiles) dérivées des statistiques de match.

**Gestion de projet (420-VB9-LP) :** la planification Agile (Kanban, MVP tagging, PRD) et la rigueur documentaire (ADR — *Architecture Decision Records*) démontrent la capacité à mener un projet complexe en solo sur plusieurs mois.

## 13 Analyse de conception : Cycle de vie et cheminement de pensée de l'onglet Live Dashboard

### 13.1 Phase 1 : Identification des contraintes d'API et architecture de contournement

La conception du *Live Dashboard* a débuté par une hypothèse de travail intuitive : interroger l'API individuellement pour chacun des 12 joueurs d'une partie, en récupérant leurs métadonnées de profil à la volée. Cette approche s'est révélée inapplicable dès la phase de prototypage. Le endpoint unitaire `GET /v1/matches/{id}/metadata` impose un *rate limiting* sévère de **3 requêtes par heure**, rendant mécaniquement impossible l'interrogation successive de 12 joueurs dans une fenêtre de temps acceptable.

Face à cette contrainte structurelle, le raisonnement architectural a conduit à explorer les endpoints alternatifs disponibles dans la documentation de l'API. La solution identifiée est le *bulk metadata endpoint* : `GET /v1/matches/metadata?match_ids={id}`, conçu précisément pour centraliser en une seule requête l'ensemble des métadonnées d'une partie. Ce endpoint offre un quota de **10 requêtes par minute**, soit une amélioration d'un facteur 200 sur la fenêtre d'utilisation, et opère sans dépendance directe aux services Steam, éliminant les erreurs 503 intermittentes qui affectaient le endpoint unitaire. Ce choix illustre un principe fondamental de conception logicielle : avant d'optimiser, il faut identifier et remettre en question les hypothèses de départ.

### 13.2 Phase 2 : Design asynchrone et gestion d'état de l'interface

La seconde difficulté architecturale concernait la réactivité de l'interface. Les requêtes réseau vers l'API Deadlock, dont les temps de réponse peuvent dépasser 800 ms, auraient gelé le fil d'exécution principal du renderer Chromium si elles avaient été exécutées de manière synchrone. Ce gel aurait produit une interface non réactive et une expérience utilisateur dégradée, particulièrement perceptible sur les animations de chargement.

La solution adoptée repose sur une **machine à états explicite** gouvernant le cycle de vie de l'onglet. Trois états distincts ont été définis et implémentés :

1. **Idle / Loading** : état initial déclenché dès que l'utilisateur active l'onglet. L'interface affiche un *shimmer effect* — une animation CSS de squelette de contenu — sur chaque tuile de joueur, signalant visuellement que les données sont en cours de récupération sans bloquer l'interaction.
2. **Mutation / Parsing** : état de traitement, activé à la réception de la réponse brute de l'API. Un normaliseur TypeScript transforme le schéma du *bulk endpoint* (dont les champs `team`, `game_mode` et `winning_team` diffèrent du schéma unitaire) vers le type interne `RichMatchMeta`. Ce typage strict garantit que les composants d'interface ne reçoivent jamais de données partielles ou mal formées.
3. **Final / Rendered** : état terminal, déclenché après l'hydratation complète des données normalisées. Les tuiles de joueurs sont peuplées, les tags dynamiques sont appliqués via le `DynamicTagManager`, et les Awards de la partie sont calculés et affichés.

Cette séparation claire des états élimine toute ambiguïté dans les transitions d'interface et facilite le débogage : chaque état est observable et traçable indépendamment des autres.

### 13.3 Phase 3 : Gestion défensive des cas d'erreur (Fallbacks)

La troisième phase de conception a porté sur la résilience de l'onglet face aux défaillances de l'API. *Deadlock-API.com*, en tant qu'API communautaire non officielle, est sujette à des interruptions de service, des erreurs 503 (*Service Unavailable*) et des temps de réponse erratiques. Concevoir un *Live Dashboard* qui refuse d'afficher quoi que ce soit en cas d'erreur aurait constitué une expérience utilisateur inacceptable.

La stratégie de **mode dégradé honnête** a été mise en place autour de la persistance locale via **electron-store**. Lors de chaque chargement réussi d'une partie, les métadonnées normalisées sont écrites en cache avec un **TTL de 7 jours**. En cas d'erreur 503 ou d'absence de connectivité lors d'une requête ultérieure, le système bascule automatiquement vers ces données en cache, avec un indicateur visuel clair informant l'utilisateur que les données affichées proviennent du dernier chargement réussi et peuvent ne pas être à jour.

Cette approche respecte deux principes fondamentaux : la *transparence* envers l'utilisateur, qui sait que les données sont issues du cache, et la *continuité de service*, l'application restant fonctionnelle et utile même en cas de défaillance externe. Elle constitue une application directe des notions de persistance et de gestion de base de données étudiées dans le cadre du cours 420-V91-LP.

## 14 Bilan des apprentissages réalisés

### 14.1 Apprentissages techniques

**Architecture multi-processus Electron.** Le concept le plus structurant du projet a été de comprendre et de maîtriser l'architecture en trois couches d'Electron : processus principal (Node.js, accès système), preload script (pont de sécurité via **contextBridge**), renderer (interface web Chromium). Chaque message entre le renderer et le main doit passer par un canal IPC explicitement déclaré : cette rigueur m'a appris à concevoir des interfaces logicielles compartimentées et sécurisées.

**Gestion d'une API instable et non officielle.** Travailler avec une API communautaire en développement actif m'a appris à anticiper les ruptures : documenter chaque décision architecturale (ADR), mettre en place des fallbacks, et concevoir des normaliseurs robustes capables de gérer plusieurs versions d'un même schéma. C'est une compétence directement transférable en milieu professionnel.

**Développement cross-platform Windows ↔ Linux.** Porter l'application de Windows vers Linux/CachyOS/Wayland a mis en lumière des différences profondes : chemins de fichiers, gestion des fenêtres (Wayland vs X11), permissions de processus. J'ai appris à écrire du code conditionnel robuste et à tester sur les deux plateformes.

**Pipeline OAuth complet (PKCE).** Implémenter le flux PKCE pour Spotify — génération du *code verifier*, serveur HTTP local temporaire pour le callback, gestion du refresh token, renouvellement automatique avant expiration — m'a donné une compréhension concrète des protocoles d'authentification modernes.

**Python subprocess et communication NDJSON.** Isoler le worker OCR (EasyOCR, PyTorch) dans un processus Python séparé, communiquant avec Electron via **stdout** en JSON, m'a appris le principe d'isolation des dépendances lourdes : le renderer reste réactif

pendant que le worker traite des images en arrière-plan.

**Moteur d'awards — logique algorithmique sur données réelles.** Concevoir 48 métriques de performance et modéliser leurs seuils a nécessité de raisonner rigoureusement sur les données disponibles : quels champs existent dans l'API, quelles approximations sont honnêtes, et comment communiquer les limites à l'utilisateur.

```

1 // Metrique de base : valeur divisee par la duree en minutes
2 function perMin(value: number, durationS: number): number {
3   return durationS > 0 ? value / (durationS / 60) : 0;
4 }
5
6 // Approximation du killstreak : intervalles entre morts du joueur
7 function approxKillstreak(
8   player: RichMetaPlayer,
9   durationS: number,
10 ): number | null {
11   if (!player.death_details) return null; // donnee absente
12   if (player.death_details.length === 0) return player.kills;
13
14   const snaps = player.stats; // snapshots temporels cumulatifs
15   const deaths = [...player.death_details]
16     .sort((a, b) => a.game_time_s - b.game_time_s);
17
18   // Construire les intervalles "sans mort"
19   const intervals: Array<[number, number]> = [
20     [0, deaths[0].game_time_s],
21     ...deaths.slice(0, -1).map((d, i): [number, number] =>
22       [d.game_time_s, deaths[i + 1].game_time_s]),
23     [deaths[deaths.length - 1].game_time_s, durationS],
24   ];
25
26   let maxKills = 0;
27   for (const [start, end] of intervals) {
28     const w = (snapAtOrBefore(snaps, end)?.kills ?? 0)
29       - (snapAtOrBefore(snaps, start)?.kills ?? 0);
30     if (w > maxKills) maxKills = w;
31   }
32   return maxKills; // approximation conservative (snapshots 3-6 min)
33 }

```

Listing 4: Moteur des awards — calcul par minute et approximation de killstreak

La logique de seuils des awards peut se représenter comme un arbre de décision :

La figure 8 illustre le cas archétypal des awards : une seule métrique scalaire comparée à des seuils hiérarchisés. Deux autres familles méritent une analyse approfondie, car leur évaluation implique une logique de calcul sensiblement plus complexe. Dans les deux cas, les données brutes arrivent sous forme de l'objet `RichMatchMeta` : la réponse JSON de l'endpoint `GET /v1/matches/metadata?match_ids={id}` est transmise du processus principal au renderer via le canal IPC (`ipcRenderer.invoke('api:request', ...)`), puis désérialisée et passée directement à `computeAwardsForMatch`.

### Famille Kill Participation — double condition simultanée.

Les awards de participation aux éliminations imposent une **double condition conjonctive** : le joueur doit simultanément dépasser un seuil absolu d'actions (20 kills + assists

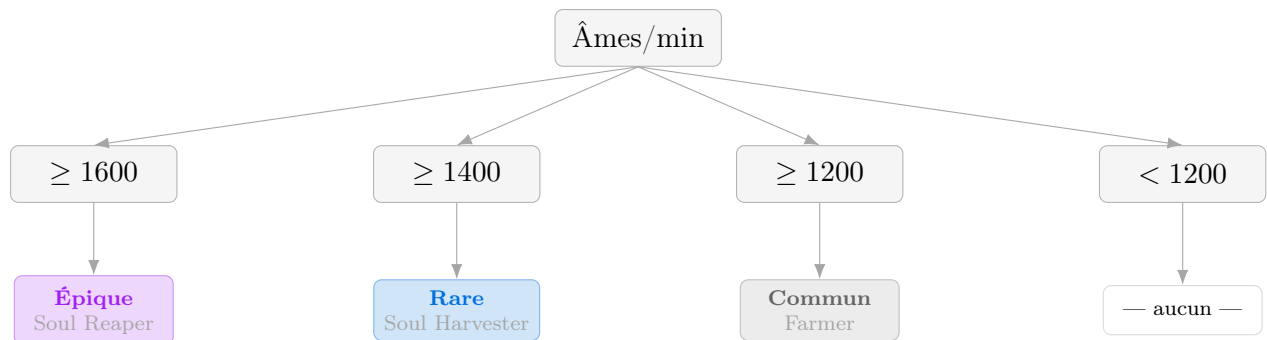


Figure 8: Arbre de décision des awards basés sur les âmes par minute

cumulés) *et* un pourcentage du total de kills de son équipe. La figure 9 illustre cet arbre à deux niveaux de décision.

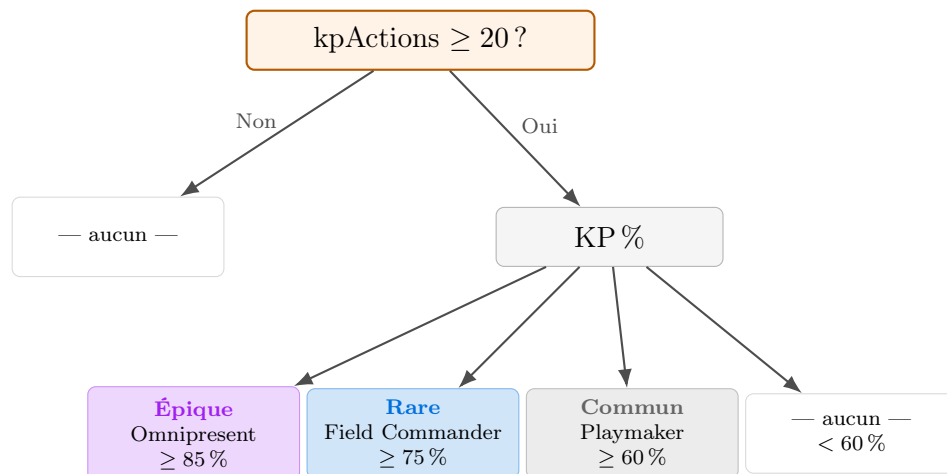


Figure 9: Arbre de décision des awards Kill Participation — porte d'entrée absolue (kpActions) et seuils de pourcentage hiérarchisés.

Un comportement important de cette famille est la **cumulabilité des awards** : les trois conditions sont des blocs **if** séquentiels indépendants, et non une structure **else if**. Un joueur atteignant 85 % de KP avec 25 actions décroche simultanément *Omnipresent*, *Field Commander* et *Playmaker*. Le champ `teamKills` n'est pas fourni directement par l'API ; il est recalculé dans le `render` en filtrant `meta.players` sur `p.team === player.team` et en sommant les kills : un calcul local qui garantit la cohérence avec le payload reçu sans requête réseau supplémentaire.

### Famille Killstreak — approximation par intervalles inter-morts.

Les awards de killstreak (*On Fire* 10, *Heat Wave* 15, *Blaze of Glory* 20) sont les plus algorithmiquement complexes du moteur. L'API Deadlock ne fournit aucun horodatage individuel par élimination : la série maximale doit être **approximée** à partir de deux structures présentes dans `RichMatchMeta`. La figure 10 illustre le flux de calcul complet, de la réception des données à l'attribution des awards.

L'approximation présente un **biais conservateur intentionnel** : les snapshots `stats[]` sont capturés toutes les 3 à 6 minutes, et non à chaque kill. Si un joueur enchaîne 8 kills

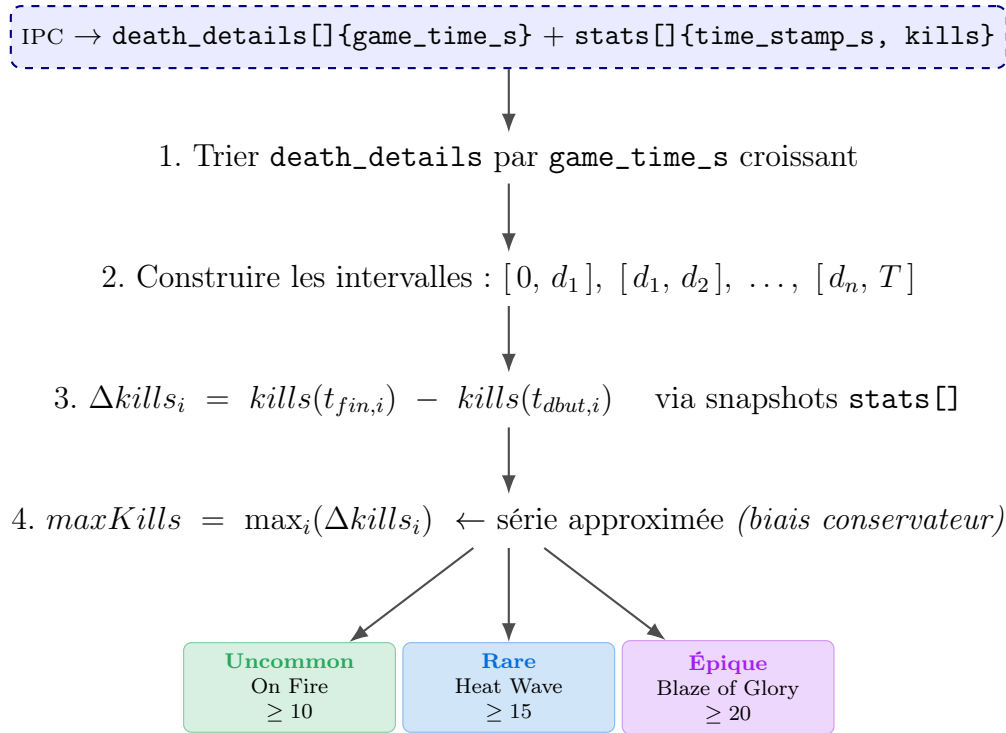


Figure 10: Diagramme de flux de l’approximation du killstreak — intervalles inter-morts et snapshots cumulatifs `stats[]`, reçus via IPC.

dans un intervalle de 90 secondes entre deux morts rapprochées, mais que le snapshot le plus proche date d’avant cet intervalle, seuls les kills visibles dans la plage temporelle du snapshot seront comptés. L’award n’est accordé que si le seuil est atteint malgré cette sous-estimation : c’est une décision délibérée qui garantit qu’aucun award de killstreak n’est décerné à tort.

Deux cas limites critiques sont gérés explicitement : (1) si `death_details` est absent du payload (certains matchs archivés n’exposent pas ce champ), la fonction retourne `null` et les trois awards sont entièrement ignorés pour ce match plutôt que d’afficher une valeur erronée ; (2) si `death_details.length === 0` (partie parfaite sans aucune mort), l’intégralité de `player.kills` constitue une seule séquence ininterrompue et est retournée directement, sans construction d’intervalles. Ces deux garde-fous illustrent le même principe de programmation défensive que `Math.max(1, stats.deaths)` dans `DynamicTagManager` : préférer un résultat absent à un résultat incorrect.

Le module `DynamicTagManager`, présenté ci-après, illustre un second réinvestissement de ces mêmes compétences, appliqué cette fois à la classification comportementale des joueurs en temps réel.

```

1 // src/services/TagManager.ts
2 export interface PlayerPerformance {
3   kills: number;
4   deaths: number;
5   assists: number;
6   netWorth: number;
7   damageMitigated: number;
8   objectiveDamage: number;
9   gameDurationMin: number;

```

```

10 }
11
12 export type PlaystyleTag =
13     | 'GLASS_CANNON'
14     | 'OBJECTIVE_BEAST'
15     | 'TACTICAL_FEEDER'
16     | 'GOLD_MINER';
17
18 export class DynamicTagManager {
19     public static generateTags(
20         stats: PlayerPerformance,
21     ): PlaystyleTag[] {
22         const tags: PlaystyleTag[] = [];
23
24         // Math.max(1, deaths) : programmation défensive contre la
25         // division
26         // par zéro lors d'une partie parfaite (0 mort)
27         const kda =
28             (stats.kills + stats.assists) /
29             Math.max(1, stats.deaths);
30         const gpm = stats.netWorth / stats.gameDurationMin;
31         const objPerMin =
32             stats.objectiveDamage / stats.gameDurationMin;
33
34         if (kda > 3.5 && stats.deaths > stats.kills * 1.2) {
35             tags.push('GLASS_CANNON');
36         }
37         if (objPerMin > 150) {
38             tags.push('OBJECTIVE_BEAST');
39         }
40         if (gpm > 1100) {
41             tags.push('GOLD_MINER');
42         }
43         if (kda < 1.0 && (stats.kills + stats.assists) > 15) {
44             tags.push('TACTICAL_FEEDER');
45         }
46
47         return tags;
48     }
49 }

```

Listing 5: DynamicTagManager — classification comportementale par métriques de performance

Ce module démontre le réinvestissement concret des notions d’algorithmique et de statistiques quantitatives du DEC. Les métriques calculées — KDA, GPM et dégâts d’objectif par minute — sont des ratios normalisés qui permettent de comparer des joueurs indépendamment de la durée de la partie, un raisonnement directement issu des cours de mathématiques appliquées (201-V03-LP) et d’algèbre et statistiques (201-V04-LP).

Un aspect particulièrement représentatif de la **programmation défensive** est l’expression `Math.max(1, stats.deaths)` au calcul du KDA. Sans cette protection, une partie parfaite — où un joueur terminerait avec **zéro mort** — provoquerait une division par zéro, produisant un résultat `Infinity` ou `NaN` en JavaScript. Ces valeurs se propagent silencieusement dans les calculs suivants, corrompent les comparaisons conditionnelles (`Infinity > 3.5` évalue à `true`, ce qui déclencherait faussement le tag `GLASS_CANNON`), et peuvent ultimement

faire crasher un composant d'interface en arrière-plan. Le `Math.max(1, stats.deaths)` ramène le dénominateur à 1 au minimum, garantissant que le résultat reste un nombre fini et sémantiquement cohérent quel que soit le scénario de jeu, y compris les cas limites statistiquement rares mais tout à fait légitimes.

**Rédaction d'un rapport professionnel en L<sup>A</sup>T<sub>E</sub>X.** La production de ce document officiel m'a initié à la typographie professionnelle avec L<sup>A</sup>T<sub>E</sub>X : maîtrise des environnements (`lstlisting` pour le code, `tikzpicture` pour les diagrammes, `figure` pour les captures d'écran, `tabular` pour les tableaux), gestion des flottants, configuration de Babel pour le français, et génération automatique de la table des matières et de la liste des figures. Cette rigueur de mise en page est directement applicable à la rédaction de documentation technique professionnelle.

## 14.2 Apprentissages en gestion de projet et outils

**Obsidian et Microsoft Planner comme colonne vertébrale.** L'utilisation combinée d'Obsidian (notes structurées, diagrammes Excalidraw, liens bidirectionnels entre concepts) et de Microsoft Planner (Kanban, tags MVP vs. Backlog) m'a appris à *externaliser* la complexité : plutôt que de garder les décisions en tête, je les matérialise dans des outils dédiés. Cette habitude réduit la charge cognitive, améliore la qualité des décisions et produit une traçabilité naturelle de l'évolution du projet.

**Claude Code, skills et MCP servers — orchestrer l'IA comme outil professionnel.** Ce projet m'a permis de développer une compétence émergente : savoir « orchestrer » un assistant IA de développement. Au-delà de la génération de code, j'ai appris à utiliser des *skills* spécialisés : `/grill-me` pour valider un plan technique face à des questions précises, `/grill-with-docs` pour confronter une décision à la documentation et aux ADR existants, et `/latex-handouts` pour produire ce rapport officiel. Les *MCP servers* (*Model Context Protocol*) m'ont appris à donner à l'IA un accès structuré au projet : fichiers, historique Git, documentation, afin d'obtenir des réponses contextualisées plutôt que génériques. Apprendre à formuler le bon contexte, à valider les réponses et à itérer rapidement est une compétence professionnelle que ce projet m'a permis de développer concrètement.